Type Casting

Type Casting : Type Casting is the process of changing the value of one data type to another data type is called Type Casting.

- Casting in python is done using constructor functions, which are built-in functions used to convert one data type to another.

Common constructor Functions :

1. int() → Converts value to integer
2. float() → Converts value to float
3. str() → Converts value to string

1.Integer : int() is a built-in constructor function used to convert a value into an integer (whole number).it can take numbers, strings, or Boolean values and return an integer representation.

Example :

```
x = int(1)
y = int(2.8)
z = int("3")
print(x,y,z)
```

output : 1 2 3

After executing the program, if you're not sure whether it has been converted to an integer or not, you can check it using the "type()" function.

```
x = int(1)
y = int(2.8)
z = int("3")
print(type(x),type(y),type(z))
```

output : <class 'int'> <class 'int'> <class 'int'>

2.Float : float() is a built-in constructor function used to convert a value into a floating point numbers(decimal number type).

Example :

```
x = float(10)
y = float(4.8)
z = float("5")
print(x,y,z)
```

output : 10.0 4.8 5.0

String : str() is a built-in constructor function used to convert any data type into a string (text format).

Example :

```
x = str("s1")
y = str(4.8)
z = str(3.0)
print(x,y,z)
```

output : s1 4.8 3.0

Code Challenge : Casting

Test your understanding of Python type casting by completing a small coding challenge

Instructions:

- Inside the editor, complete the following steps:
- Create a variable x with the integer value 1
- Convert x to a float and store it in a
- Convert x to a string and store it in b
- Print a and b

Example :

```
x = int(1)
a = float(x)
b = str(x)
print(type(a),type(b))
print(a,b)
```

output : <class 'float'> <class 'str'>
1

Python Lists

List : A list is a built-in data types in python used to store multiple values in a single variable. It is ordered, changeable (mutable), and allows duplicate values.

Syntax :

list_name = [item1, item2, item3]

Example :

```
list_1 = ["one","two"]
print(list_1)
```

output : ['one', 'two']

Ordered : when we say that lists are ordered, it means that the items have a defined order, and that order will not change.

- If you add new items to a list , the new items will be placed at the end of the list.

Note: There are some list methods that will change the order, but in general: the order of the items will not change.

Changeable : The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.

Built-in Functios in List:

1. **len() :** len() is a built-in function used to return the total number of items (length) in an object such as a list, string, tuple, or dictionary.

Syntax :

len(object)

Example :

```
ls = [1,2,3,4,5]
print(len(ls))
```

output: 5

2. **type() :** type() is a built-in function used to identify and return the data type of a variable or value.

Syntax :

type(object)

Example :

```
x = (1)
y = ("hi")
z = (2.4)
print(type(x),type(y),type(z))
print(x,y,z)
```

output : <class 'int'> <class 'str'> <class 'float'>
hi 2.4

3. **sum() :** sum() is a built-in function used to add all the elements in an iterable (like a list or tuple) and return the total.

Syntax :

sum(variable)

Example :

```
x = [1,2,3]
print(sum(x))
print(x)
```

output : 6
[1, 2, 3]

4. **append()** : append() is a built-in list method used to add a single element to the end of an existing list.it modifies the original list and increases its size by one.

Syntax :

list_name.append(item)

Example :

```
x = [1,2,3]
print(x)
x.append(4)
print(x)
```

output : [1, 2, 3]
[1, 2, 3, 4]

5. **insert() :** insert() is a list method used to add an element at a specific index(Position) in the list without replacing existing elements.

Syntax :

list_name.insert(index,item)

Example :

```
my_list = [1, 3]
my_list.insert(4, 4)
print(my_list)
```

output : [1, 3, 4]

6. **count() :** count() is a list method used to return the total number of times a specified value appears in the list.

Syntax :

List_1.count(value)

Example :

```
my_ls= [1,2,3,4,2,2,3]
print(my_ls.count(2))
```

output : 3

7. **pop() :** pop() is a list method used to remove and return the element at a specified index from the list.

Syntax :

list_1.pop(index)

- if no index is given, it removes the last item

Example :

```
my_ls= [1,2,3,4,2,2,3]
my_ls.pop(1)
print(my_ls)
```

output : [1, 3, 4, 2, 2, 3]

8. **remove() :** it is used to delete the first occurrence of a specified value from the list.

Syntax :

list_1.remove(value)

Example :

```
my_ls= [1,2,3,4,2,2,3]
my_ls.remove(4)
print(my_ls)
```

output : [1, 2, 3, 2, 2, 3]

9. **sort() :** it is used to arrange the elements of a list in ascending(default) or descending order.

Syntax :

list_1.sort()

Example :

```
my_ls= [1,2,3,4,2,2,3]
my_ls.sort()
print(my_ls)
```

output : [1, 2, 2, 2, 3, 3, 4]

10. **Access :** Accessing items means retrieving(getting)individual elements from a data structure(like a list, tuple or string) using their position or key.

Syntax :

list[index]

Index start from the 0(zero).

Example :

```
x = [1,3,4,5]
print(x[0])
```

output : 1

11. **Changing :** it means updating or modifying the value of an existing element in a data structure like a list or dictionary.

Syntax :

list_1[index] = new_value

Example :

```
x = [1,3,4,5]
x[0] = 10
print(x)
```

output : [10, 3, 4, 5]

Code Challenge : List

Test your understanding of creating, accessing, changing, adding, and removing list items.

Instructions

- Inside the editor, complete the following steps:
- Create a list called colors with the values "red", "green", "blue"
- Print the first item in the list
- Change the second item to "yellow"
- Add "purple" to the end of the list using append()
- Remove "red" from the list using remove()
- Print the list

Example :

```
colors = ["red","green","blue"]
print(colors[0])
colors[1]= "yellow"
print(colors)
colors.append("purpule")
colors.remove("red")
print(colors)
```

output : red

['red', 'yellow', 'blue']

['yellow', 'blue', 'purpule']

Difference Between List and Tuple

Feature	**List**	**Tuple**
Syntax	[]	()
Mutable	Yes	No
Ordered	Yes	Yes
Duplicate Values	Allowed	Allowed
Performance	Slower	Faster
Methods	Many	Few

Sets

Set : A set is a built-in data type used to store multiple unique elements in a single variable. It is unordered, unindexed, and does not allow duplicate values.

Key Characteristics

- Unordered → items have no fixed position.
- Unindexed → Cannot access items using index.
- No duplicates → Each value appears only once.
- Mutable → You can add or remove items.

Syntax :

set_name ={item1,item2}

Example :

```
set_name = {1,3,4,5}
print(set_name)
```

output : {1, 3, 4, 5}

- Here we can see some built-in like
  1. len()
  2. type()
  3. add()
  4. update()
  5. remove()
  6. discard()
  7. pop()
  8. clear()

1. **len() :** len() is a built-in function used to return the number of elements in a set (or any iterable)**.**

Syntax :

len(object)

Example :

```
set_name = {1,3,4,5}
print(len(set_name))
```

output : 4

2. **type()** : type() returns the data type of a variable or object.

Syntax :

type(object)

Example :

```
set_name = {1,3,4,5}
print(type(set_name))
```

output : <class 'set'>

3. **add() :** add() is a set method used to add a single element to a set**.**

Syntax :

set_1.add(element)

Example:

```
set_name = {1,3,4,5}
set_name.add(2)
print(set_name)
```

output : {1, 2, 3, 4, 5}

4. **update() :** update() adds multiple elements from another iterable to the set.

Syntax :

set_1.update(iterable)

Example :

```
set_name = {1,3}
set_name.update([2,6])
print(set_name)
```

output : {1, 2, 3, 6}

5. **remove() :** remove() is used to deletes a specific elements from the set.

Syntax :

set_1.remove(element)

Example :

```
set_name = {1,2,3,2}
set_name.remove(2)
print(set_name)
```

output : {1, 3} #raises error if element is not found

6. **discard() :** it is used to removes an element without raising an error if it doesn't exist.

Syntax :

set_1.discard(element)

Example :

```
set_name = {1,2,3,2}
set_name.discard(5)
print(set_name)
```

output : {1, 2, 3}

7. **pop() :** it is used to removes and returns a random element from the set.

Syntax :

set_1.pop()

Example :

```
set_name = {1,2,3,20,40}
set_name.pop()
print(set_name)
```

output : {2, 3, 20, 40}

8. **clear() :** it removes all elements from the set.

Syntax :

set_1.clear()

Example :

```
set_name = {1,2,3,20,40}
set_name.clear()
print(set_name)
```

output: set() # set becomes empty

Code Challenge : Sets

Test your understanding of creating, adding, removing, and checking set items.

Instructions

Inside the editor, complete the following steps:

- Create a set called colors with the values **"red"**, **"green"**, **"blue"**
- Print the set
- Add **"yellow"** to the set using add()
- Remove **"green"** from the set using discard()
- Print the number of items using len()

Example :

```
colors = {"red","green","blue"}
print(colors)
colors.add("yellow")
colors.discard("green")
print(colors)
print(len(colors))
```

output : {'red', 'blue', 'green'}
{'yellow', 'red', 'blue'}
3

Dictionary in Python

Introduction:

A **Dictionary** is one of the most important built-in data structures in Python. It is used to store data in the form of **key-value pairs**. Dictionaries are highly efficient for storing, retrieving, updating, and managing data.

In real-world applications, dictionaries are widely used to represent structured information such as student records, employee details, product information, API responses, configuration settings, and JSON data.Unlike lists and tuples that use indexes to access data, dictionaries use **keys**.

Definition: A Dictionary is a mutable, ordered collection that stores data as key-value pairs.

Each key in a dictionary must be unique, and each key is associated with a value.

Syntax:

```
dictionary_name = {
    key1: value1,
    key2: value2,
    key3: value3
}
```

Example

```
student = {
    "name": "Priya",
    "age": 22,
    "course": "Python"
}
print(student)
```

Output:{'name': 'Priya', 'age': 22, 'course': 'Python'}

Why Do We Need Dictionaries?

Suppose we want to store information about a student.

Without Dictionary:

```
name = "Priya"
age = 22
course = "Python"
city = "Visakhapatnam"
```

Managing many variables becomes difficult.

Using Dictionary:

```
student = {
    "name": "Priya",
    "age": 22,
    "course": "Python",
    "city": "Visakhapatnam"
}
```

All related information is stored in a single object.

Characteristics of Dictionary

1. **Stores Data as Key-Value Pairs:**

```
student = {
   "name": "Priya",
   "age": 22
}
```

Here:

name → key

Priya → value

age → key

22 → value

2. **Ordered Collection:** Python preserves the insertion order of items.

```
data = {
    "A": 1,
    "B": 2,
    "C": 3
}
print(data)
```

Output:{'A': 1, 'B': 2, 'C': 3}

3. **Mutable:** Dictionary values can be modified.

```
student = {
    "name": "Priya",
    "age": 22
}
student["age"] = 23
print(student)
```

Output:{'name': 'Priya', 'age': 23}

4. **Duplicate Keys Not Allowed**

```
student = {
    "name": "Priya",
    "name": "Rahul"
}
print(student)
```

Output:{'name': 'Rahul'}

The last value overwrites the previous one.

5. **Duplicate Values Allowed:**

```
student = {
    "student1": "Priya",
    "student2": "Priya"
}
print(student)
```

Output:{'student1': 'Priya', 'student2': 'Priya'}

Creating Dictionaries

Method 1: Using Curly Braces

```
student = {
    "name": "Priya",
    "age": 22
}
```

Method 2: Using dict()

```
student = dict(
    name="Priya",
    age=22,
    course="Python"
)
print(student)
```

Output:{'name': 'Priya', 'age': 22, 'course': 'Python'}

Accessing Dictionary Items

Using Square Brackets

```
student = {
    "name": "Priya",
    "age": 22
}
print(student["name"])
```

Output:Priya

get() Method: The get() method returns the value of the specified key.
If the key does not exist, it returns None instead of generating an error.

Syntax

dictionary.get(key)

Example

```
student = {
    "name": "Priya"
}
print(student.get("name"))
```

Output:Priya

Dictionary Length:

len() Function: Returns the number of key-value pairs present in a dictionary.

Example

```
student = {
    "name": "Priya",
    "age": 22,
    "course": "Python"
}
print(len(student))
```

Output:3

Adding Items: New items can be added using a new key.

Example

```
student = {
    "name": "Priya"
}
student["age"] = 22
print(student)
```

Output{'name': 'Priya', 'age': 22}

Updating Items: Existing values can be modified using their keys.

Example

```
student = {
    "name": "Priya",
    "age": 22
}
student["age"] = 25
print(student)
```

Output:{'name': 'Priya', 'age': 25}

update() Method: Updates existing values or adds new values.

Syntax

dictionary.update({key:value})

Example

```
student = {
    "name": "Priya"
}
student.update({
    "age": 22,
```

```
    "course": "Python"
})
print(student)
```
Output:{'name': 'Priya', 'age': 22, 'course': 'Python'}

Removing Dictionary Items

pop() Method: Removes a specific key and returns its value.

Example

```
student = {
    "name": "Priya",
    "age": 22
}
student.pop("age")
print(student)
```
Output:{'name': 'Priya'}

popitem() Method: Removes the last inserted item.

Example

```
student = {
    "name": "Priya",
    "age": 22
}
student.popitem()
print(student)
```
Output:{'name': 'Priya'}

del Keyword: Used to delete a specific key or the entire dictionary.

Example

```
student = {
    "name": "Priya",
    "age": 22
}
del student["age"]
print(student)
```
Output:{'name': 'Priya'}

clear() Method?: Removes all items from the dictionary.

Example

```
student = {
    "name": "Priya",
    "age": 22
}
student.clear()
print(student)
```
Output:{}

Dictionary Methods

keys(): Returns all keys.

```
student = {
    "name": "Priya",
    "age": 22
```

```
}
print(student.keys())
```
Output: dict_keys(['name', 'age'])

values(): Returns all values.

```
print(student.values())
```
Output: dict_values(['Priya', 22])

items() : Returns all key-value pairs as tuples.

```
print(student.items())
```
Output: dict_items([('name', 'Priya'), ('age', 22)])

copy(): Creates a copy of the dictionary.

```
new_student = student.copy()
print(new_student)
```
setdefault(): Returns the value of a key.

If the key does not exist, inserts the key with the specified value.

Example

```
student = {
    "name": "Priya"
}
student.setdefault("age", 22)
print(student)
```
Output:{'name': 'Priya', 'age': 22}

Looping Through Dictionary

Loop Through Keys:

```
for key in student:
    print(key)
```
Loop Through Values:

```
for value in student.values():
    print(value)
```
Loop Through Keys and Values:

```
for key, value in student.items():
    print(key, value)
```
Output:name Priya

age 22

Nested Dictionary: A dictionary inside another dictionary is called a nested dictionary.

Example

```
students = {
    101: {
        "name": "Priya",
        "age": 22
    },
    102: {
        "name": "Rahul",
        "age": 23
    }
}
```

```
print(students)
```

Dictionary Comprehension: Dictionary comprehension provides a short and elegant way to create dictionaries.

Syntax

```
{
    key:value
    for item in iterable
}
```

Example

```
square = {
    x: x*x
    for x in range(1,6)
}
print(square)
```

Output: {1:1, 2:4, 3:9, 4:16, 5:25}

Dictionary Functions

Function	Description
len()	Returns number of items
type()	Returns object type
max()	Returns largest key
min()	Returns smallest key
sorted()	Sorts dictionary keys
dict()	Creates dictionary

Tuple in Python

Introduction:

A **Tuple** is a built-in data type in Python used to store multiple items in a single variable.

Tuples are similar to lists, but the main difference is that **tuples are immutable**, which means their values cannot be changed after creation.

Tuples are used when data should remain fixed and protected from modification.

Examples include storing coordinates, employee records, student information, dates, and configuration settings.

Definition: A Tuple is an ordered and immutable collection of elements.

Tuples can store different data types such as integers, strings, floats, lists, and even other tuples.

Syntax:

tuple_name = (item1, item2, item3)

Example

```
student = ("Priya", 22, "Python")
print(student)
```

Output
('Priya', 22, 'Python')

Why Do We Need Tuples?

Without Tuple:

```
name = "Priya"
age = 22
course = "Python"
```

Managing multiple variables becomes difficult.

Using Tuple:

```
student = ("Priya", 22, "Python")
```

All related information is stored in one variable.

Characteristics of Tuple

1. Ordered Collection:

Tuples maintain the order of elements.

```
colors = ("Red", "Green", "Blue")
print(colors)
```

Output
('Red', 'Green', 'Blue')

2. Immutable:

Once a tuple is created, its values cannot be modified.

```
student = ("Priya", 22)
student[1] = 23
```

Output
TypeError: 'tuple' object does not support item assignment

3.Allows Duplicate Values:

```
numbers = (10, 20, 10, 30)
print(numbers)
```

Output
(10, 20, 10, 30)

4. Supports Multiple Data Types:

```
data = ("Priya", 22, 95.5, True)
print(data)
```

Output
('Priya', 22, 95.5, True)

Creating Tuples

Method 1: Using Parentheses

```
fruits = ("Apple", "Mango", "Orange")
```

Method 2: Using tuple() Constructor

```
fruits=tuple(("Apple","Mango","Orange"))
print(fruits)
```

Output
('Apple', 'Mango', 'Orange')

Tuple Length

len() Function : Returns the number of elements present in the tuple.

Example

```
fruits = ("Apple", "Mango", "Orange")
print(len(fruits))
```

Output

3

Accessing Tuple Elements

Using Index : Tuple elements can be accessed using index numbers.

Example

```
fruits = ("Apple", "Mango", "Orange")
print(fruits[0])
```

Output

Apple

Negative Indexing : Negative indexes start from the end.

Example

```
fruits = ("Apple", "Mango", "Orange")
print(fruits[-1])
```

Output

Orange

Tuple Slicing : Slicing is used to retrieve a range of elements.

Syntax

tuple[start:end]

Example

```
numbers = (10, 20, 30, 40, 50)
print(numbers[1:4])
```

Output

(20, 30, 40)

Updating Tuples : Since tuples are immutable, values cannot be changed directly.

Example

```
student = ("Priya", 22)
student[1] = 23
```

Output

TypeError

Converting Tuple to List :

To modify a tuple, first convert it to a list.

Example

```
student = ("Priya", 22)
temp = list(student)
temp[1] = 23
student = tuple(temp)
print(student)
```

Output

('Priya', 23)

Adding Elements to Tuple :

Tuples cannot be directly modified.

Example

```
fruits = ("Apple", "Mango")
fruits = fruits + ("Orange",)
print(fruits)
```

Output

('Apple', 'Mango', 'Orange')

Deleting Tuple Elements :

Individual elements cannot be removed.

However, the entire tuple can be deleted.

Example

```
fruits = ("Apple", "Mango", "Orange")
del fruits
```

Tuple Packing : Assigning multiple values into a tuple is called tuple packing.

Example

```
student = ("Priya", 22, "Python")
```

Tuple Unpacking : Extracting tuple values into separate variables is called tuple unpacking.

Example

```
student = ("Priya", 22, "Python")
name, age, course = student
print(name)
print(age)
print(course)
```

Output

Priya

22

Python

Tuple Methods :

Tuples have only two built-in methods.

count() : Returns the number of occurrences of a value.

Syntax :

tuple.count(value)

Example

```
numbers = (10, 20, 10, 30, 10)
print(numbers.count(10))
```

Output

3

index() : Returns the index position of the first occurrence of a value.

Syntax :

tuple.index(value)

Example

```
numbers = (10, 20, 30, 40)
print(numbers.index(30))
```

Output

2

Looping Through Tuples

Using for Loop

```
fruits = ("Apple", "Mango", "Orange")
for item in fruits:
    print(item)
```

Output

Apple

Mango

Orange

Using while Loop

```
fruits = ("Apple", "Mango", "Orange")
i = 0
while i < len(fruits):
    print(fruits[i])
    i += 1
```

Nested Tuples : A tuple inside another tuple is called a nested tuple.

Example

```
students = (
    ("Priya", 22),
    ("Rahul", 23),
    ("Kiran", 24)
)
print(students)
```

Output

(('Priya', 22), ('Rahul', 23), ('Kiran', 24))

Membership Operators :

in Operator : Checks whether an element exists.

Example

```
fruits = ("Apple", "Mango", "Orange")
print("Mango" in fruits)
```

Output

True

not in Operator :

Example

```
print("Banana" not in fruits)
```

Output

True

Built-in Functions Used with Tuples

max() : Returns largest value.

```
numbers = (10, 20, 30, 40)
```

```
print(max(numbers))
```
Output
40
min(): Returns smallest value.
```
numbers = (10, 20, 30, 40)
print(min(numbers))
```
Output
10
sum() : Returns total of all elements.
```
numbers = (10, 20, 30)
print(sum(numbers))
```
Output
60
sorted() : Returns a sorted list.
```
numbers = (40, 10, 30, 20)
print(sorted(numbers))
```
Output
[10, 20, 30, 40]
Advantages of Tuple :

- Faster than lists
- Uses less memory
- Immutable and secure
- Can be used as dictionary keys
- Suitable for fixed data

Disadvantages of Tuple:

- Cannot modify elements
- Cannot add or remove items directly
- Limited built-in methods

Real-Time Applications of Tuples :
Storing Coordinates
point = (10, 20)
Student Records
student = ("Priya", 22, "Python")
Database Records
employee = (101, "Rahul", 50000)
Returning Multiple Values from Functions
```
def details():
    return "Priya", 22
name, age = details()
print(name)
print(age)
```
Output
Priya
22